CS & IT ENGINEERING

'C' Programming

Pointers & Arrays

Lecture No.- 03

By- Satya sir

Recap of Previous Lecture

- Pointer datatype
- Pointer size
- Void Pointer
- NULL Pointer
- Constant Pointer
- Arrays
  - 1-D array

Topics to be Covered

- Initialization of 1-D array
- Array Programs
- String handling in 'C'

1-D Array Initialization

- Along with declaration

Ex:

```
int x[5] = {10, 20, 30};

Printf("%d", x[3]); // 0
Printf("%d", x[4]); // 0
```

x

0	1	2	3	4
10	20	30	0	0

```
int x[5];

Printf("%d %d %d", x[0], x[2], x[4]);

// Garbage values
```

```
int x[5] = {0};
```

x

0	1	2	3	4
0	0	0	0	0

Topic : String Handling in 'C'

$$\texttt{int x[5]} = \{10,20,30,40,50,\underbrace{60,70}_{\text{ignored}}\};$$

	0	1	2	3	4
X	10	20	30	40	50

$$\texttt{int x[ ]} = \{\underbrace{10,20,30,40,50,60,70}\}$$

size → == 7

	0	1	2	3	4	5	6
X	10	20	30	40	50	60	70

```
int x[ ];
// Error, array size must be declared
```

NOTE: Arrays are always allocated Static Memory.
– Compile-time memory allocation.

```
int x[5] = {10, 20, 30};
```

	0	1	2	3	4
X	10	20	30	0	0

```
printf("%d", x[6]);
x[2] = x[3] + x[7];
```
} Access leads to Error.

// Find Minimum Element from array

```
#include<stdio.h>
int main( )
{
  int x[20], n, min, i;
  Printf(" Enter No.of.Elements");
  scanf(" %d", &n);
  for( i=0; i<n; i++ )
     scanf(" %d", &x[i]);
   min=x[0];
   for(i=1; i<n; i++)
   { if( x[i]<min)
        min=x[i];
   }
Printf("The minimum Element is %d", min);
 return 0;
}
```

For Maximum Element

```
if( x[i]>max )
   max=x[i];
```

Topic : String Handling in 'C'

Strings :

– Group of characters (or) character array (or) character Pointer also.

Syntax : `char stringname [size] ;`

`char x[20];`

Initialization : `char x[ ] = { 'G', 'A', 'T', 'E', 'E', 'x', 'A', 'M' };`

0	1	2	3	4	5	6	7
G	A	T	E	E	x	A	M

x

(OR)

`Char x[ ] = "GATEEXAM";`

0	1	2	3	4	5	6	7	8
G	A	T	E	E	x	A	M	\0

x

\0 → NULL Character

Ex:

```
char str[5];
Printf("%c", str[0]);
Printf("%c", str[2]);
```

} Garbage values

```
Char str[5] = {'x'};
```

str

0	1	2	3	4
x	\0	\0	\0	\0

```
Printf("%c %c", str[2], str[4]);
```

O/P : No output

```c
char str[5] = {'C'};
```

	0	1	2	3	4
str	C	\0	\0	\0	\0

```c
printf("%d %d %d", str[0], str[2], str[4]);
```

o/p: 67 0 0

'C' ASCII Value   '\0' ASCII Value

```c
int x[5];
for(i=0; i<5; i++)
    scanf("%d", &x[i]);
```

```c
char x[5];
scanf("%s", x);
```

```c
int x[5], y[5];
for(i=0; i<5; i++)
    scanf("%d %d", &x[i], &y[i]);
```

```c
char x[5], y[5];
scanf("%s %s", x, y);
// Accepts only 1 i/p.
```

Topic : String Handling in 'C'

strlen( )

Syntax: Strlen(string)

Strlen(): returns the index of first NULL character (\0) if Exists, otherwise size of string.

```
char X[5] = {'A','B','C','D','E'};
char Y[5] = "ABCDE";
Printf("%d", strlen(X));   #5
Printf("%d", strlen(Y));   #5
Printf("%d", Sizeof(X));   #5
Printf("%d", Sizeof(Y));   #6
```

X:

0	1	2	3	4
A	B	~~C~~ \0	D	E

Y:

0	1	2	3	4	5
A	B	~~C~~ \0	D	E	\0

'\0' Indicates End of String

Sizeof( )

⇒ returns the amount of memory allocated in Bytes.

```
X[2] = '\0';  Y[2] = '\0';
Printf("%d", strlen(X));   #2
Printf("%d", strlen(Y));   #2
Printf("%d %d", sizeof(X), sizeof(Y));
// 5 6
```

Topic : String Handling in ‘C’

No.	Function	Description
1)	strlen(string_name)	returns the length of string name.
2)	strcpy(destination, source)	copies the contents of source string to destination string.
3)	strcat(first_string, second_string)	concats or joins first string with second string. The result of the string is stored in first string.
4)	strcmp(first_string, second_string)	compares the first string with second string. If both strings are same, it returns 0.
5)	strrev(string)	returns reverse string.
6)	strlwr(string)	returns string characters in lowercase.
7)	strupr(string)	returns string characters in uppercase.
8)	strstr(str1, str2)	It returns a pointer to the first occurrence of the given substring str2 within the given string str1

Topic : String Handling in 'C'

Please read and go through

No.	Function	Description
9)	strncmp()	It compares two strings only to n characters.
10)	strncat()	It concatenates n characters of one string to another string.
11)	strncpy()	It copies the first n characters of one string into another.
12)	strchr()	It finds out the first occurrence of a given character in a string.
13)	strrchr()	It finds out the last occurrence of a given character in a string.
14)	strnstr()	It finds out the first occurrence of a given string in a string where the search is limited to n characters.
15)	strcasecmp()	It compares two strings without sensitivity to the case.
16)	strncasecmp()	It compares n characters of one string to another without sensitivity to the case.

- strlen( )
- strcat( )
- strcmp( )
- strlwr( )
- strupr( )
- strchr( )
- strcpy( )

2 mins Summary

- 1-D array Initialization
- Strings
- String vs Number arrays

To be Contd . . . ☺

THANK - YOU